

```

1  """
2  HNSW is an approximate nearest-neighbor algorithm used by vector databases
3  and Azure AI Search.
4
5  The ingestion pipeline:
6  1. Defines the Chunk model
7  2. Defines the search-index schema
8  3. Creates the Azure AI Search index
9  4. Receives document text
10 5. Creates chunks
116. Embeds chunks in batches
127. Attaches embeddings and metadata
138. Uploads everything to the index
14"""
15
16from azure.core.credentials import AzureKeyCredential
17from azure.search.documents import SearchClient
18from azure.search.documents.indexes import SearchIndexClient
19from azure.search.documents.indexes.models import (
20    HnswAlgorithmConfiguration,
21    SearchField,
22    SearchFieldDataType,
23    SearchIndex,
24    SearchableField,
25    SimpleField,
26    VectorSearch,
27    VectorSearchProfile,
28)
29from openai import OpenAI
30from pydantic import BaseModel, Field, field_validator
31
32
33class Chunk(BaseModel):
34    text: str
35    chunk_id: str
36    chunk_type: str
37    doc_id: str
38    parent_id: str | None = None
39    page_number: int | None = None
40    embedding: list[float] | None = None
41
42
43def define_index_fields(vector_dim: int):
44    fields = [
45        SimpleField(
46            name="chunk_id",
47            type=SearchFieldDataType.String,
48            key=True,
49            filterable=True,
50        ),
51        SimpleField(
52            name="doc_id",
53            type=SearchFieldDataType.String,
54            filterable=True,
55        ),
56        SimpleField(
57            name="chunk_type",
58            type=SearchFieldDataType.String,
59            filterable=True,
60        ),
61        SimpleField(
62            name="parent_id",
63            type=SearchFieldDataType.String,
64            filterable=True,
65    ],

```

```

66         SimpleField(
67             name="page_number",
68             type=SearchFieldDataType.Int32,
69             filterable=True,
70         ),
71         SimpleField(
72             name="row_number",
73             type=SearchFieldDataType.Int32,
74             filterable=True,
75         ),
76         SearchableField(
77             name="text",
78             type=SearchFieldDataType.String,
79         ),
80         SearchField(
81             name="embedding",
82             type=SearchFieldDataType.Collection(
83                 SearchFieldDataType.Single
84             ),
85             searchable=True,
86             vector_search_dimensions=vector_dim,
87             vector_search_profile_name="vector-profile",
88         ),
89     ]
90
91     return fields
92
93
94 def create_search_index(endpoint, key, index_name, vector_dim):
95     client = SearchIndexClient(
96         endpoint=endpoint,
97         credential=AzureKeyCredential(key),
98     )
99
100     vector_search = VectorSearch(
101         algorithms=[
102             HnswAlgorithmConfiguration(name="hnsw-config")
103         ],
104         profiles=[
105             VectorSearchProfile(
106                 name="vector-profile",
107                 algorithm_configuration_name="hnsw-config",
108             )
109         ],
110     )
111
112     index = SearchIndex(
113         name=index_name,
114         fields=define_index_fields(vector_dim),
115         vector_search=vector_search,
116     )
117
118     return client.create_or_update_index(index)
119
120
121 class DocumentInput(BaseModel):
122     document_id: str
123     text: str = Field(min_length=1, max_length=2000)
124     strategy: str = Field(default="flat", min_length=1)
125
126     @field_validator("strategy")
127     @classmethod
128     def strategy_validator(cls, value: str) -> str:
129         value = " ".join(value.split()).lower()
130

```

```

131         if value in ("hierarchical", "flat"):
132             return value
133
134         raise ValueError("strategy must be flat or hierarchical")
135
136     @field_validator("text")
137     @classmethod
138     def text_validator(cls, value: str) -> str:
139         value = " ".join(value.split())
140
141         if not value:
142             raise ValueError("text must not be empty")
143
144         return value
145
146
147 client = OpenAI()
148
149
150 def embed_fn(texts):
151     response = client.embeddings.create(
152         model="text-embedding-3-small",
153         input=texts,
154     )
155
156     return [item.embedding for item in response.data]
157
158
159 def embed_chunks(
160     chunks,
161     embedding_fn,
162     text_key="text",
163     batch_size=64,
164 ):
165     embedded_chunks = []
166
167     for i in range(0, len(chunks), batch_size):
168         batch = chunks[i:i + batch_size]
169         texts = [chunk[text_key] for chunk in batch]
170         embeddings = embedding_fn(texts)
171
172         for chunk, embedding in zip(batch, embeddings):
173             new_chunk = chunk.copy()
174             new_chunk["embedding"] = embedding
175             embedded_chunks.append(new_chunk)
176
177     return embedded_chunks
178
179
180 def upload_chunks(
181     endpoint,
182     key,
183     index_name,
184     embedded_chunks,
185 ):
186     search_client = SearchClient(
187         endpoint=endpoint,
188         index_name=index_name,
189         credential=AzureKeyCredential(key),
190     )
191
192     return search_client.upload_documents(
193         documents=embedded_chunks
194     )
195

```

```
196
197 def embed_query(query, embed_fn):
198     return embed_fn([query])[0]
```